P0907R3
Signed Integers are Two's Complement

Published Proposal, 10 June 2018

This version:
http://wg21.link/p0907r3

Issue Tracking:
Inline In Spec

Author:
JF Bastien (Apple)

Audience:
CWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/p0907r3.bs

Abstract

There is One True Representation for signed integers, and that representation is two's complement.

Table of Contents

§ 1. Introduction

[C11] Integer types allows three representations for signed integral types:

- Signed magnitude
- Ones' complement
- Two’s complement

See §5 C Signed Integer Wording for full wording.

C++ goes further than C and only requires that "the representations of integral types shall define values by use of a pure binary numeration system". To the author’s knowledge no modern machine uses both C++ and a signed integer representation other than two’s complement (see §6 Survey of Signed Integer Representations). None of [MSVC], [GCC], and [LLVM] support other representations. This means that the C++ that is taught is effectively two’s complement, and the C++ that is written is two’s complement. It is extremely unlikely that there exist any significant codebase developed for two’s complement machines that would actually work when run on a non-two’s complement machine.

C and C++ as specified, however, are not two’s complement. Signed integers currently allow the existence of an extraordinary value which traps, extra padding bits, integral negative zero, and introduce undefined behavior and implementation-defined behavior for the sake of this *extremely* abstract machine.

Let’s stop pretending that the abstract machine should represent integers as signed magnitude or ones' complement. These theoretical implementations are a different programming language, not our real-world C++. Developers who require signed magnitude or ones' complement integers would be better served by a pure-library solution, and so would the rest of us.

This paper proposes the following:

- *Status-quo* Signed integer arithmetic remains non-commutative in general (though some implementations may guarantee that it is).
- *Change* `bool` is represented as `0` for `false` and `1` for `true`. All other representations are undefined.
- *Change* `bool` only has value bits, no padding bits.
- *Change* Signed integers are two’s complement.
- *Change* If there are *M* value bits in the signed type and *N* in the unsigned type, then *M = N-1* (whereas C says $M \leq N$).
- *Status-quo* If a signed operation would naturally produce a value that is not within the range of the result type, the behavior is undefined. The author had hoped to make this well-defined as wrapping (the operations produce the same value bits as for the corresponding unsigned type), but WG21 had strong resistance against this.
- *Change* None of the integral types have extraordinary values.
- *Change* C11 note 53 has wording aroung trap representations within padding bits, e.g. for parity bits. C++ has no such wording.
- *Change* Conversion from signed to unsigned is always well-defined: the result is the unique value of the destination type that is congruent to the source integer modulo 2^N.
- *Change* Conversion from enumeration type to integral is the same as that of converting from the enumeration’s underlying type and then to the destination type, even if the original value cannot be represented by the specified type.
- *Status-quo* Conversion from integral type to enumeration is unchanged: if the original value is not within the range of the enumeration values the behavior is undefined.
- *Change* Left-shift on signed integer types produces the same results as left-shift on the corresponding unsigned integer type.

- *Change* Right-shift is an arithmetic right shift which performs sign-extension.
- *Status-quo* shift by larger-than or equal-to bit-width remains undefined.
- *Status-quo* `constexpr` evaluation of signed integer arithmetic with undefined result is not a *core constant expression*.
- *Change* the range of enumerations without a fixed underlying type is simplified because of two's complement.
- *Status-quo* `is_modulo` type trait remains `false` for signed integer types unless an implementation chooses to defined overflow as wrap.
- `has_unique_object_representations<T>` is `true` for `bool` and signed integer types, in addition to unsigned integer types and others before.
- *Status-quo* atomic operations on signed integer types continues not to have undefined behavior, and is still specified to wrap as two's complement (the definition is clarified to act as-if casting to unsigned and back).
- *Change* address [LWG3047] to remove undefined behavior in pre-increment atomic operations.

This proposal leaves C unchanged, it merely restricts further the subset of C which applies to C++. Aaron Ballman volunteered to present this paper and the corresponding [N2218] to WG14, in the hope that C will approve compatible changes. The WG14 feedback is summarized in §7 WG14 Feedback from the Brno Meeting.

A final argument to move to two's complement is that few people spell "ones' complement" correctly according to Knuth [TAoCP]. Reducing the nerd-snipe potential inherent in C++ is a Good Thing™.

> Detail-oriented readers and copy editors should notice the position of the apostrophe in terms like "two's complement" and "ones' complement": A two's complement number is complemented with respect to a single power of 2, while a ones' complement number is complemented with respect to a long sequence of 1s. Indeed, there is also a "twos' complement notation," which has radix 3 and complementation with respect to $(2 \ldots 22)_3$.

§ 2. Edit History

§ 2.1. r2 → r3

Add §3.1 Annex C (informative) Compatibility.

As requested in Jacksonville, the paper was presented to EWG a second time in Rapperswil with feedback from WG14. Reception was extremely positive, and the paper was forwarded to CWG through the following poll.

	SF	***F***	***N***	***A***	***SA***
Forward to Core	*4*	*17*	*0*	*0*	*0*

§ 2.2. r1 → r2

Update with feedback from WG14, available in §7 WG14 Feedback from the Brno Meeting.

§ 2.3. r0 → r1

This paper was presented in Jacksonville to:

- A joint SG6 numerics and SG12 undefined behavior session
- The Evolution Working Group

The following polls were taken, and corresponding modifications made to the paper. The main change between [P0907r0] and the subsequent revision is to maintain undefined behavior when signed integer overflow occurs, instead of defining wrapping behavior. This direction was motivated by:

- Performance concerns, whereby defining the behavior prevents optimizers from assuming that overflow never occurs;
- Implementation leeway for tools such as sanitizers;
- Data from Google suggesting that over 90% of all overflow is a bug, and defining wrapping behavior would not have solved the bug.

It is expected that this paper have little to no effect on code generation from current compilers. Known codegen changes should have no performance implication, for example:

- Older x86 vector instructions don’t support arithmetic right shift, and therefore must remain scalar instead of vectorizing to non-arithmetic right shift.
- An implementation which implements left shift using a rotate now need to also mask.

2.3.1. SG6 / SG12

	SF	*F*	*N*	*A*	*SA*
Change anything with respect to signed integers.	*2*	*10*	*5*	*1*	*0*
Moving any change from this paper to IS'20 is blocked on synchronizing with WG14.	*4*	*9*	*4*	*0*	*1*
Change the default unsigned integral types' behavior.	*0*	*0*	*1*	*8*	*9*
Changes we make constrain extended integral types.	*10*	*4*	*2*	*0*	*1*
Allow extraordinary value for signed integral types.	*2*	*2*	*9*	*2*	*3*

We decided to independently conside what the "privileged" syntax' behavior should be when going out of range for:

- cast of enums
- cast from `unsigned` to `signed`
- `INT_MIN / -1`
- addition / subtraction / multiplication and `-INT_MIN` overflow

Multiple answer poll would you be OK with these being the standards-mandated behavior for the "privileged" syntax behavior of signed integral types when going out of range:

- 16—undefined behavior
- 0—saturation
- 0—trap (exception)
- 1—trap (abort)
- 1—generate extraordinary value
- 11—wrap
- 5—Rust-style wrap or trap, maybe contract violation?
- 9—intermediate values are mathematical integers (e.g. `(int)a+(int)b > INT_MAX` would be OK)

	SF	*F*	*N*	*A*	*SA*

Cast to enums outside of the enum's representable range should be defined instead of undefined behavior.	*0*	*1*	*1*	*5*	*8*
Cast from `unsigned` to `signed` which are out of range is currently implementation defined. It should instead be wrap.	*6*	*6*	*3*	*0*	*1*
We want two's complement representation for signed types, regardless of what WG14 decides.	*7*	*4*	*2*	*0*	*3*
`INT_MIN / -1` should be defined.	*0*	*0*	*4*	*3*	*9*

Multiple answer poll addition / subtraction / multiplication and `-INT_MIN` overflow is currently undefined behavior, it should instead be:

- 4—wrap
- 6—wrap or trap
- 5—intermediate values are mathematical integers
- 14—status quo (remain undefined behavior)

Multiple answer poll to opt-in to the other behaviors (both for `signed` and `unsigned`) we can create library or language changes. What should we explore separately from this paper?

- 12—undefined behavior
- 9—saturation
- 8—trap (exception)
- 6—trap (abort)
- 3—generate extraordinary value
- 16—wrap
- 10—Rust-style wrap or trap, maybe contract violation?
- 9—intermediate values are mathematical integers (e.g. `(int)a+(int)b > INT_MAX` would be OK)
- 13—bigint
- 14—operations with a bool set when overflow has occurred

	SF	***F***	***N***	***A***	***SA***

Shifting out of range `1 << 31` as currently defined (if shift has defined value when interpreted as unsigned, you get that value as signed, you can shift into but not past the sign, shifting past sign bit is undefined behavior). `1 << 31` was defined and still is (as of Howard's paper), `2 << 31` was undefined behavior and still is. WG14 is currently considering whether to adopt Howard's paper which made this. Should we take it back to undefined to do `1 << 31`?	*2*	*7*	*5*	*1*	*1*
Left shift should be the same for `signed` and `unsigned` (overrides previous poll).	*6*	*5*	*5*	*0*	*1*
Right shift on a signed integral type should be an arithmetic shift (which sign-extends).	*9*	*4*	*3*	*1*	*0*
In C `intN_t` is optional: "These types are optional. However, if an implementation provides integer types with widths of 8, 16, 32, or 64 bits, no padding bits, and (for the signed types) that have a two's complement representation, it shall define the corresponding typedef names." Change it to make `int8_t`, `int16_t`, `int32_t`, and `int64_t` not optional.	*1*	*0*	*2*	*3*	*12*

2.3.2. EWG

The Evolution Working Group took the following polls:

	SF	***F***	***N***	***A***	***SA***
Disallow extraordinary value (trapping / NaN) for signed integral types.	*16*	*12*	*7*	*1*	*2*
Does EWG want to move signed integers to two's complement, as presented in the current paper (without extraordinary values)?	*11*	*17*	*7*	*2*	*1*
Move to Core.	*7*	*8*	*12*	*8*	*4*

The resolution on disallowing extraordinary values overrides the lack of consensus for change from SG6 / SG12.

The decision to not forward to Core was mainly motivated on hearing back from WG14. WG14 met in Brno, discussed [N2218], and provided feedback detailed in §7 WG14 Feedback from the Brno Meeting.

§ 3. Proposed Wording

Leave the note in Program execution [**intro.execution**] ¶8 as-is:

> [*Note:* Operators can be regrouped according to the usual mathematical rules only where the operators really are associative or commutative. For example, in the following fragment
>
> ```
> int a, b;
> /* ... */
> a = a + 32760 + b + 5;
> ```
>
> the expression statement behaves exactly the same as
>
> ```
> a = (((a + 32760) + b) + 5);
> ```
>
> due to the associativity and precedence of these operators. Thus, the result of the sum `(a + 32760)` is next added to `b`, and that result is then added to 5 which results in the value assigned to `a`. On a machine in which overflows produce an exception and in which the range of values representable by an `int` is `[-32768, +32767]`, the implementation cannot rewrite this expression as
>
> ```
> a = ((a + b) + 32765);
> ```
>
> since if the values for `a` and `b` were, respectively, -32754 and -15, the sum `a + b` would produce an exception while the original expression would not; nor can the expression be rewritten either as
>
> ```
> a = ((a + 32765) + b);
> ```
>
> or
>
> ```
> a = (a + (b + 32765));
> ```
>
> since the values for `a` and `b` might have been, respectively, 4 and -8 or -17 and 12. However on a machine in which overflows do not produce an exception and in which the results of overflows are reversible, the above expression statement can be rewritten by the implementation in any of the above ways because the same result will occur. —*end note*]

Modify Fundamental types [**basic.fundamental**] ¶4 onwards:

Unsigned integers shall obey the laws of arithmetic modulo 2^n where *n* is the number of bits in the value representation of that particular size of integer.

> This implies that unsigned arithmetic does not overflow because a result that cannot be represented by the resulting unsigned integer type is reduced modulo the number that is one greater than the largest value that can be represented by the resulting unsigned integer type.

Type `wchar_t` is a distinct type whose values can represent distinct codes for all members of the largest extended character set specified among the supported locales. Type `wchar_t` shall have the same size, signedness, and alignment requirements as one of the other integral types, called its *underlying type*. Types `char16_t` and `char32_t` denote distinct types with the same size, signedness, and alignment as `uint_least16_t` and `uint_least32_t`, respectively, in `<cstdint>`, called the underlying types.

Values of type `bool` are either `true` or `false`[†]. [*Note:* There are no `signed`, `unsigned`, `short`, or `long bool` types or values. —*end note*] Values of type `bool` participate in integral promotions. Type `bool`'s object representation bits shall only contain value bits. The bits of the value representation of type `bool` shall all be zero for `false`. For `true`, all bits shall be zero except for the bit corresponding to the least-significant bit in the object representation of the unsigned type with the same size (or a bit-field thereof) as `bool`. The program has undefined behavior if any other object representation is formed.

> [†] Using a `bool` value in ways described by this International Standard as "undefined", such as by examining the value of an uninitialized automatic object, might cause it to behave as if it is neither `true` nor `false`.

> ISSUE 1 We need to define the storage for `bool` since we define signed and unsigned below, and that excludes `bool`. The definition I propose is more restrictive than what we had before because it only allows two values to be represented, and doesn't allow padding bits. This guarantees that `bool` is trivially-copyable, and gives it a unique object representation, which as far as I know all compilers already guaranteed.

Types `bool`, `char`, `char16_t`, `char32_t`, `wchar_t`, and the signed and unsigned integer types are collectively called *integral* types. A synonym for integral type is *integer type*. ~~The representations of integral types shall define values by use of a pure binary numeration system[‡]. [*Example:* This document permits two's complement, ones' complement and signed magnitude representations for integral types. —*end example*]~~

For unsigned integer types, the bits of the object representation are divided into two groups: value bits and padding bits. There need not be any padding bits; `unsigned char` shall not have any padding bits. If there are *N* value bits, each bit shall represent a different power of 2 between 1 and 2^{N-1}, so that objects of that type shall be capable of representing values from 0 to $2^{N}-1$ using a pure binary numeration system[‡]; this shall be known as the value representation. The values of any padding bits are unspecified.

> [‡] A positional representation for integers that uses the binary digits 0 and 1, in which the values represented by successive bits are additive, begin with 1, and are multiplied by successive integral power of 2, except perhaps for the bit with the highest position. (Adapted from the *American National Dictionary for Information Processing Systems*.)

ISSUE 2 An intermediate revision of this paper stated "Value bits are store contiguously in memory" in an attempt to preserve different endiannesses, and otherwise restricts implementations to "sane" layout. However, that change wasn't presented to EWG and received pushback on the SG6 reflector. Would this be a desirable addition, should it be in a separate paper, or does it overconstrain implementations?

Signed integer types shall be repesented as two's complement. The bits of signed integer types' object representation are divided into three groups: value bits, padding bits, and the sign bit. There need not be any padding bits; `signed char` shall not have any padding bits. There shall be exactly one sign bit. Each bit that is a value bit shall have the same value as the same bit in the object representation of the corresponding unsigned type (if there are *M* value bits in the signed type and *N* in the unsigned type, then *M = N-1*). The value bit in the unsigned representation with value 2^{N-1} corresponds to the sign bit in the signed representation, where it has value -2^{N-1}. [*Note:* Unless otherwise specified, if a signed operation would naturally produce a value that is not within the range of the result type, the behavior is undefined (see [**expr.pre**] 8). —*end note*]

ISSUE 3 note *M = N-1*, whereas C says $M \leq N$. I derive this from "For each of the standard signed integer types, there exists a corresponding (but different) standard unsigned integer type [...] **each of which occupies the same amount of storage**".

All combinations of value bits and sign bit are valid and result in a specific distinct integer value. No other bits factor into an integer's value. [*Note:* Therefore, none of the integral types have extraordinary values as defined in ISO C. —*end note*]

> ISSUE 4 In C11 implementation requirements call it the "extraordinary value" and refer to 6.2.6.2 which calls it "is a trap representation or a normal value". Furthermore, in note 53 there's wording around extraordinary values being held in padding bits (not just as a value stolen from the value representation), and since C++ wording used to only talk about "binary representation" it'd rather be very clear about the absence of extraordinary values. It's unclear whether SG6 / SG12 guidance explicitly wanted to disallow padding bits from being special and trapping. I propose disallowing it.

Modify Integral conversions [**conv.integral**] ¶1 onwards:

> A prvalue of an integer type can be converted to a prvalue of another integer type. A prvalue of an unscoped enumeration type can be converted to a prvalue of an integer type.
>
> ~~If the destination type is unsigned, the resulting value is the least unsigned integer congruent to the source integer (modulo 2^n where *n* is the number of bits used to represent the unsigned type). [*Note:* In a two's complement representation, this conversion is conceptual and there is no change in the bit pattern (if there is no truncation). —*end note*] If the destination type is signed, the value is unchanged if it can be represented in the destination type; otherwise, the value is implementation-defined.~~
>
> For integral types other than `bool`, the result is the unique value of the destination type that is congruent to the source integer modulo 2^N, where *N* is the number of value bits in the destination type. [*Note:* This conversion is conceptual; there is no change in the bit pattern other than that high-order bits may either be discarded or added. All added high-order bits will have the same value. —*end note*]

Modify Static cast [**expr.static.cast**] ¶9 onwards:

A value of a scoped enumeration type can be explicitly converted to an integral type ~~. When that type is *cv* `bool`, the resulting value is `false` if the original value is zero and `true` for all other values. For the remaining integral types, the value is unchanged if the original value can be represented by the specified type. Otherwise, the resulting value is unspecified.~~ ; the result is the same as that of converting from the enumeration's underlying type and then to the destination type. A value of a scoped enumeration type can also be explicitly converted to a floating-point type; the result is the same as that of converting from the original value to the floating-point type.

> ISSUE 5 the above change is for enumeration *to* integer, which SG6 did not object to changing as suggested. It states the conversion in terms of the updated [**conv.integral**] rules.

A value of integral or enumeration type can be explicitly converted to a complete enumeration type. If the enumeration type has a fixed underlying type, the value is first converted to that type by integral conversion, if necessary, and then to the enumeration type. If the enumeration type does not have a fixed underlying type, the value is unchanged if the original value is within the range of the enumeration values, and otherwise, the behavior is undefined. A value of floating-point type can also be explicitly converted to an enumeration type. The resulting value is the same as converting the original value to the underlying type of the enumeration, and subsequently to the enumeration type.

> ISSUE 6 the above unmodified paragraph is for integer *to* enumeration which SG6 voted to leave undefined in poll "Cast to enums outside of the enum's representable range should be defined instead of undefined behavior".

Modify Shift operators [**expr.shift**] ¶1 onwards:

> The operands shall be of integral or unscoped enumeration type and integral promotions are performed. The type of the result is that of the promoted left operand. The behavior is undefined if the right operand is negative, or greater than or equal to the length in bits of the promoted left operand.
>
> The value of `E1 << E2` is `E1` left-shifted `E2` bit positions; vacated bits are zero-filled. ~~If `E1` has an unsigned type, the~~ The value of the result is $E1 \times 2^{E2}$, reduced modulo ~~one more than the maximum value representable in the result type. Otherwise, if `E1` has a signed type and non-negative value, and $E1 \times 2^{E2}$ is representable in the corresponding unsigned type of the result type, then that value, converted to the result type, is the resulting value; otherwise, the behavior is undefined.~~ 2^N, where *N* is the number of value bits in the result type.
>
> ISSUE 7 updated to reflect poll "Left shift should be the same for signed and unsigned".
>
> The value of `E1 >> E2` is `E1` right-shifted `E2` bit positions. ~~If `E1` has an unsigned type or if `E1` has a signed type and a non-negative value, the~~ The value of the result is ~~the integral part of the quotient of `E1`/2^{E2}. If `E1` has a signed type and a negative value, the resulting value is implementation-defined.~~ `E1 / 2`^{E2}, rounded down. [*Note*: Right-shift on signed integral types is an arithmetic right shift, which performs sign-extension. —*end note*]

Leave Constant expressions [**expr.const**] ¶2 as-is:

> An expression `e` is a *core constant expression* unless the evaluation of `e`, following the rules of the abstract machine, would evaluate one of the following expressions:
>
> […]
>
> - an operation that would have undefined behavior as specified in Clause 4 through 19 of this document [*Note*: including, for example, signed integer overflow, certain pointer arithmetic, division by zero, or certain shift operations —*end note*]

Modify Enumeration declarations [**dcl.enum**] ¶8:

> For an enumeration whose underlying type is fixed, the values of the enumeration are the values of the underlying type. Otherwise, for an enumeration where e_{min} is the smallest enumerator and e_{max} is the largest, the values of the enumeration are the values in the range b_{min} to b_{max}, defined as follows: ~~Let *K* be 1 for a two's complement representation and 0 for a ones' complement or sign-magnitude representation.~~ b_{max} is the smallest value greater than or equal to max($|e_{min}|$ - ~~K~~ 1 , $|e_{max}|$) and equal to 2^M-1, where *M* is a non-negative integer. b_{min} is zero if e_{min} is non-negative and -(b_{max}+ ~~K~~ 1) otherwise. The size of the smallest bit-field large enough to hold all the values of the enumeration type is max(M,1) if b_{min} is zero and M+1 otherwise. It is possible to define an enumeration that has values not defined by any of its enumerators. If the *enumerator-list* is empty, the values of the enumeration are as if the enumeration had a single enumerator with value 0.

Leave `numeric_limits` members [**numeric.limits.members**] ¶61 onwards as-is:

> ```
> static constexpr bool is_modulo;
> ```
>
> `true` if the type is modulo. A type is modulo if, for any operation involving `+`, `-`, or `*` on values of that type whose result would fall outside the range `[min(), max()]`, the value returned differs from the true value by an integer multiple of `max() - min() + 1`.
>
> [*Example:* `is_modulo` is `false` for signed integer types unless an implementation, as an extension to this document, defines signed integer overflow to wrap. —*end example*]
>
> Meaningful for all specializations.

Modify Type properties [**meta.unary.prop**] ¶9 as follows:

> The predicate condition for a template specialization `has_unique_object_representations<T>` shall be satisfied if and only if:
>
> - `T` is trivially copyable, and
> - any two objects of type `T` with the same value have the same object representation, where two objects of array or non-union class type are considered to have the same value if their respective sequences of direct subobjects have the same values, and two objects of union type are considered to have the same value if they have the same active member and the corresponding members have the same value.
>
> The set of scalar types for which this condition holds is implementation-defined. [*Note:* If a type has padding bits, the condition does not hold; otherwise, the condition holds true for `bool`, signed, and unsigned integral types. —*end note*]
>
> ISSUE 8 this was missing from P0907r0. Adding signed here seems like a no-brainer. GCC and LLVM currently return `true` for `bool` (and it currently isn't implemented in MSVC). Try it out.

Modify Class template `ratio` [**ratio.ratio**] ¶1 as follows:

> If the template argument `D` is zero or the absolute values of either of the template arguments `N` and `D` is not representable by type `intmax_t`, the program is ill-formed. [*Note:* These rules ensure that infinite ratios are avoided and that for any negative input, there exists a representable value of its absolute value which is positive. ~~In a two's complement representation, this~~ This excludes the most negative value. —*end note*]

Modify Specializations for integers [**atomics.types.int**] ¶7 and ¶8 as follows:

Remarks: For signed integer types, ~~arithmetic is defined to use two's complement representation.~~ the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type. [*Note:* There are no undefined results arising from the computation. —*end note*]

> ISSUE 9 the operations this applies to are add, or, and, sub, xor, and is only meaningful for add and sub.

```
T operator op=(T operand) volatile noexcept;
T operator op=(T operand) noexcept;
```

Effects: Equivalent to:

~~`return fetch_key(operand) op operand;`~~ `return static_cast<T>(static_cast<make_unsigned_t<T>>(fetch_key(operand)) op static_cast<make_unsigned_t<T>>(operand));`

> ISSUE 10 there's an outstanding defect report for this [LWG3047], whose resolution should be updated as above.

§ 3.1. Annex C (informative) Compatibility

Leave Annex C (informative) Compatibility [**diff**] as-is. A discussion on the Core reflector has determined that no updates are required with respect to C compatibility.

§ 4. Out of Scope

This proposal focuses on the representation of signed integers, and on tightening the specification when that representation is constrained to two's complement. It is out of scope for this proposal to deal with related issues which have more to them than simply the representation of signed integers.

A non-comprehensive list of items left purposefully out:

- Left and right shift with a right-hand-side equal to or wider than the bit-width of the left-hand-side.
- Integral division or modulo by zero.

- Integral division or modulo of the signed minimum integral value for a particular integral type by minus one.
- Overflow of pointer arithmetic.
- Library solution for ones' complement integers.
- Library solution for signed magnitude integers.
- Library solution for two's complement integers with trapping or undefined overflow semantics.
- Language support for explicit signed overflow truncation such as Swift's (`&+`, `&-`, and `&*`), or complementary trapping overflow operators.
- Library or language support for saturating arithmetic.
- Mechanism to let the compiler assume that integers, signed or unsigned, do not experience signed or unsigned wrapping for:
  - A specific integral variable.
  - All integral variables (à la `-ftrapv`, `-fno-wrapv`, and `-fstrict-overflow`).
  - A specific loop's induction variable.
- Mechanism to have the compiler list places where it could benefit from knowing that overflow cannot occur (à la `-Wstrict-overflow`).
- Endianness of integral storage (or endianness in general).
- Bits per bytes, though we all know there are eight.

These items could be tackled in separate proposals, unless the committee wants them tackled here. This paper expresses no preference in whether they should be addressed or how.

§ 5. C Signed Integer Wording

The following is the wording on integers from the C11 Standard.

For unsigned integer types other than unsigned char, the bits of the object representation shall be divided into two groups: value bits and padding bits (there need not be any of the latter). If there are *N* value bits, each bit shall represent a different power of 2 between 1 and 2^{N-1}, so that objects of that type shall be capable of representing values from 0 to $2^{N} - 1$ using a pure binary representation; this shall be known as the value representation. The values of any padding bits are unspecified.

For signed integer types, the bits of the object representation shall be divided into three groups: value bits, padding bits, and the sign bit. There need not be any padding bits; `signed char` shall not have any padding bits. There shall be exactly one sign bit. Each bit that is a value bit shall have the same value as the same bit in the object representation of the corresponding unsigned type (if there are *M* value bits in the signed type and *N* in the unsigned type, then M ≤ N). If the sign bit is zero, it shall not affect the resulting value. If the sign bit is one, the value shall be modified in one of the following ways:

- the corresponding value with sign bit 0 is negated (*sign and magnitude*);
- the sign bit has the value $-(2^{M})$ (*two's complement*);
- the sign bit has the value $-(2^{M} - 1)$ (*ones' complement*).

Which of these applies is implementation-defined, as is whether the value with sign bit 1 and all value bits zero (for the first two), or with sign bit and all value bits 1 (for ones' complement), is a trap representation or a normal value. In the case of sign and magnitude and ones' complement, if this representation is a normal value it is called a *negative zero*.

If the implementation supports negative zeros, they shall be generated only by:

- the `&`, `|`, `^`, `~`, `<<`, and `>>` operators with operands that produce such a value;
- the `+`, `-`, `*`, `/`, and `%` operators where one operand is a negative zero and the result is zero;
- compound assignment operators based on the above cases.

It is unspecified whether these cases actually generate a negative zero or a normal zero, and whether a negative zero becomes a normal zero when stored in an object.

If the implementation does not support negative zeros, the behavior of the `&`, `|`, `^`, `~`, `<<`, and `>>` operators with operands that would produce such a value is undefined.

The values of any padding bits are unspecified. A valid (non-trap) object representation of a signed integer type where the sign bit is zero is a valid object representation of the corresponding

> unsigned type, and shall represent the same value. For any integer type, the object representation where all the bits are zero shall be a representation of the value zero in that type.
>
> The *precision* of an integer type is the number of bits it uses to represent values, excluding any sign and padding bits. The *width* of an integer type is the same but including any sign bit; thus for unsigned integer types the two values are the same, while for signed integer types the width is one greater than the precision.

§ 6. Survey of Signed Integer Representations

Here is a non-comprehensive history of signed integer representations:

- Two’s complement
    - John von Neumann suggested use of two’s complement binary representation in his 1945 First Draft of a Report on the EDVAC proposal for an electronic stored-program digital computer.
    - The 1949 EDSAC, which was inspired by the First Draft, used two’s complement representation of binary numbers.
    - Early commercial two’s complement computers include the Digital Equipment Corporation PDP-5 and the 1963 PDP-6.
    - The System/360, introduced in 1964 by IBM, then the dominant player in the computer industry, made two’s complement the most widely used binary representation in the computer industry.
    - The first minicomputer, the PDP-8 introduced in 1965, uses two’s complement arithmetic as do the 1969 Data General Nova, the 1970 PDP-11.
- Ones' complement
    - Many early computers, including the CDC 6600, the LINC, the PDP-1, and the UNIVAC 1107.
    - Successors of the CDC 6600 continued to use ones' complement until the late 1980s.
    - Descendants of the UNIVAC 1107, the UNIVAC 1100/2200 series, continue to do so, although ClearPath machines are a common platform that implement either the 1100/2200 architecture (the ClearPath IX series) or the Burroughs large systems architecture (the ClearPath NX series). Everything is common except the actual CPUs, which are implemented as ASICs. In addition to the IX (1100/2200) CPUs and the NX (Burroughs large systems) CPU, the architecture had Xeon (and briefly Itanium) CPUs. Unisys' goal was

to provide an orderly transition for their 1100/2200 customers to a more modern architecture.

- Signed magnitude
    - The IBM 700/7000 series scientific machines use sign/magnitude notation, except for the index registers which are two's complement.

Wikipedia offers more details and has comprehensive sources for the above.

Thomas Rodgers surveyed popular DSPs and found the following:

- SHARC family ships a C++ compiler which supports C++11, and where signed integers are two's complement.
- Freescale/NXP ships a C++ compiler which supports C++03, and where signed integers are two's complement (reference).
- Texas Instruments ships a C++ compiler which supports C++14, and where signed integers are two's complement.
- Cirrus Logic (formerly Wolfson) has a C compiler only which is probably two's complement.
- CML Microcircuits has fixed ASIC for radio processing, and doesn't seem to support C++.
- Synaptics (formerly Connexant) makes audio input subsystem for voice assistants. The DSP runs fixed far-field signal processing algorithms and has programmable functions which run on standard ARM controller, using Raspbian.

In short, the only machine the author could find using non-two's complement are made by Unisys, and no counter-example was brought by any member of the C++ standards committee. Nowadays Unisys emulates their old architecture using x86 CPUs with attached FPGAs for customers who have legacy applications which they've been unable to migrate. These applications are unlikely to be well served by modern C++, signed integers are the least of their problem. Post-modern C++ should focus on serving its existing users well, and incoming users should be blissfully unaware of integer esoterica.

§ 7. WG14 Feedback from the Brno Meeting

WG14 met in Brno to discuss [N2218]. The paper was received very positively, especially given that no one in the room knew of an extant architecture that was not two's complement for which there was a reasonably modern C compiler. The closest anyone came was the Unisys ClearPath compiler documentation which says:

> Two's complement arithmetic is used on many platforms. On ClearPath MCP systems, arithmetic is performed on data in signed-magnitude form. This can cause discrepancies in algorithms that depend on the two's complement representation.

However, this compiler documentation also says that they only target C90, and was last updated on 2017.

There was worry about the search on impacted architectures not having been exhaustive. Given that WG14 will ship the IS in 5 years, it was felt that making them aware now should give vendors plenty of time to bring up reasons why this change would be bad for them.

It was pointed out that C already has cases that require two's complement.

It was noted that the existing implementation latitude is a burden for library developers because they have to consider test cases where integers are not two's complement but they have no way to actually exercise any of those test cases.

The following straw poll was taken:

	For	***Opposed***	***Abstain***
Remove (not deprecate) non-two's complement representations for signed integers.	*14*	*0*	*3*

§ References

§ Informative References

[C11]
Programming Languages — C. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

[GCC]
GCC C Implementation-Defined Behavior: Integers. URL: https://gcc.gnu.org/onlinedocs/gcc/Integers-implementation.html

[LLVM]
LLVM Language Reference Manual. URL: https://llvm.org/docs/LangRef.html

[LWG3047]
Tim Song. atomic compound assignment operators can cause undefined behavior when corresponding fetch_meow members don't. New. URL: https://wg21.link/lwg3047

[MSVC]

MSVC C Implementation-Defined Behavior: Integers. URL: https://docs.microsoft.com/en-us/cpp/c-language/integers

[N2218]

JF Bastien. Signed integers are two's complement. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2218.html

[P0907r0]

JF Bastien. Signed Integers are Two's Complement. URL: https://wg21.link/p0907r0

[TAoCP]

Donald Knuth. The Art of Computer Programming, Volume 2 (3rd Ed.): Seminumerical Algorithms.

§ Issues Index

ISSUE 1 We need to define the storage for `bool` since we define signed and unsigned below, and that excludes `bool`. The definition I propose is more restrictive than what we had before because it only allows two values to be represented, and doesn't allow padding bits. This guarantees that `bool` is trivially-copyable, and gives it a unique object representation, which as far as I know all compilers already guaranteed. ↵

ISSUE 2 An intermediate revision of this paper stated "Value bits are store contiguously in memory" in an attempt to preserve different endiannesses, and otherwise restricts implementations to "sane" layout. However, that change wasn't presented to EWG and received pushback on the SG6 reflector. Would this be a desirable addition, should it be in a separate paper, or does it overconstrain implementations? ↵

ISSUE 3 note *M = N-1*, whereas C says *M ≤ N*. I derive this from "For each of the standard signed integer types, there exists a corresponding (but different) standard unsigned integer type [...] **each of which occupies the same amount of storage**". ↵

ISSUE 4 In C11 implementation requirements call it the "extraordinary value" and refer to 6.2.6.2 which calls it "is a trap representation or a normal value". Furthermore, in note 53 there's wording around extraordinary values being held in padding bits (not just as a value stolen from the value representation), and since C++ wording used to only talk about "binary representation" it'd rather be very clear about the absence of extraordinary values. It's unclear whether SG6 / SG12 guidance explicitly wanted to disallow padding bits from being special and trapping. I propose disallowing it. ↵

ISSUE 5 the above change is for enumeration *to* integer, which SG6 did not object to changing as suggested. It states the conversion in terms of the updated [**conv.integral**] rules. ↵

ISSUE 6 the above unmodified paragraph is for integer *to* enumeration which SG6 voted to leave undefined in poll "Cast to enums outside of the enum's representable range should be defined instead of undefined behavior". ↵

ISSUE 7 updated to reflect poll "Left shift should be the same for signed and unsigned". ↵

ISSUE 8 this was missing from P0907r0. Adding signed here seems like a no-brainer. GCC and LLVM currently return `true` for `bool` (and it currently isn't implemented in MSVC). Try it out. ↵

ISSUE 9 the operations this applies to are add, or, and, sub, xor, and is only meaningful for add and sub. ↵

ISSUE 10 there's an outstanding defect report for this [LWG3047], whose resolution should be updated as above. ↵